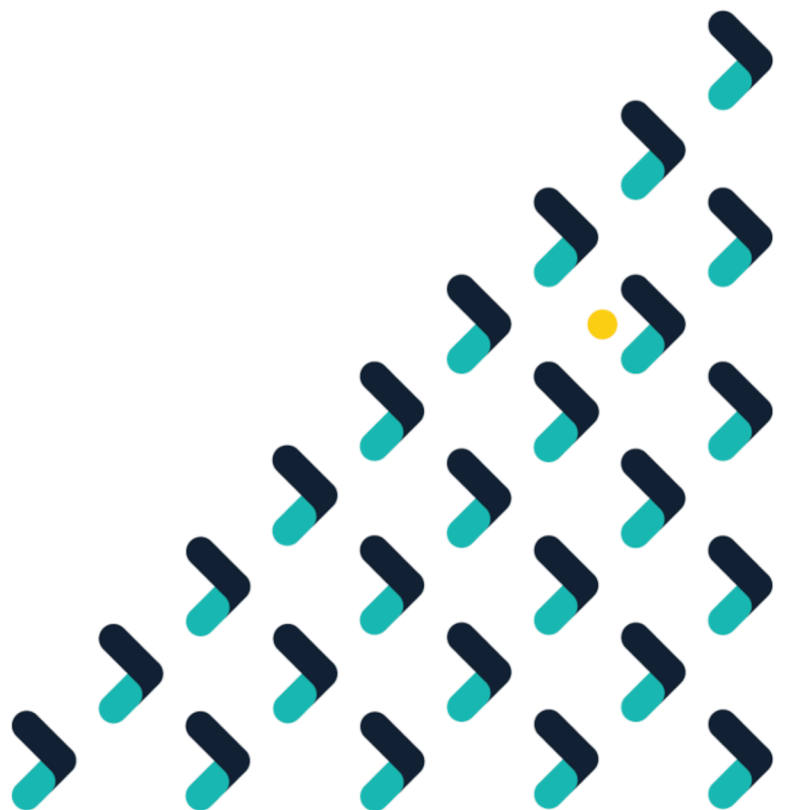




Allgemeine Richtlinien für Abas Customizing

Forterro Deutschland Abas GmbH



Inhaltsverzeichnis

Grundlegendes zur Verwendung von Programmiersprachen	1
FO2	1
Java	1
Easycode	1
FO	1
ShellScripte	1
Abweichende Programmiersprachen	2
Cronjobs	3
Grundsätzliche Strategien	3
Mehrere einzelne Cronjobs	3
Ein zentraler Cronjob mit sequentieller Abarbeitung	3
Alternative Ansätze	4
Best Practices	4
Empfohlene Vorgehensweise (je nach Szenario)	5
Zusätzliche Hinweise aus der Praxis	5
Performance und Datenmengen	6
Benennung von Variablen Datenbank bzw. Infosysteme	7
Dont's (was müssen wir vermeiden)	8
Ungenutzter Programmcode	8
Sonstige dont's	8

Grundlegendes zur Verwendung von Programmiersprachen

Es werden die beiden Programmiersprachen FO2 und Java bevorzugt. Ausnahmen sind nur erlaubt, wenn sie untenstehende Kriterien erfüllen.

FO2

Zu verwendende Standardsprache zur Umsetzung von Anforderungen mit geringer bis mittlerer Komplexität

Java

Zu verwendende Standardsprache zur Umsetzung von Anforderungen mittlerer bis sehr hoher Komplexität. Java Libraries von Dritten sind nur dann zu verwenden, wenn diese zum Auslieferungsstand von Abas gehören oder nach Freigabe durch die Teamleads Custom Development. Eine Freigabe dieser Libraries kann nur erfolgen wenn Sie lizenzrechtlich freigegeben und langfristig gewartet werden.

Easycode

Easycode wurde viele Jahre lang in den Projekten als Programmiereditor und damit als eine Art Programmiersprache verwendet. Es existieren also bei vielen IBS Kunden noch viele Easycode Programme und das Wissen über EasyCode nimmt stetig ab.

Ein neues Programm darf deswegen nicht mehr mit EasyCode erstellt werden. Es dürfen nur notwendige Änderungen und Erweiterungen in EasyCode Programmen vorgenommen werden.

Sinnvoll ist dabei die Konvertierung eines zu bearbeitenden EasyCode Programms in ein FO2 Programm. Hierbei unterstützen die abas Tools. Ein Easycode Programm kann als ETF in EasyCode exportiert werden und in den abas tools importiert (mittels File-Import-FO2 Easycode-Import ETF Files) werden.

FO

Der Einsatz von reinem FO (Spaghetti FO) ist nur dann zulässig wenn kleinste Änderungen (wenige Zeilen Code) an einem Standard FOP oder einem schon bestehenden FOP vorgenommen werden. Müssen neue Programmstrukturen in ein bestehendes FO Programm integriert werden so wird hierzu ein FO2 Programm erstellt, welches als Subprogramm aufgerufen wird.

Dies gilt auch für die Individualisierung von Standardinfosystem wie z.B. im Druck (EVVORGANG), welche in FO-Code geschrieben sind. Hier dürfen kleine Anpassungen in FO gemacht werden. Dennoch ist zu beachten, dass Programmteile die in Sub-FOP's ausgelagert werden können, in FO2 geschrieben werden müssen.

ShellScripte

Der Einsatz von Shellsripten ist möglichst zu vermeiden (außer bei Cronjobs). Shellskripte sind nur dann zulässig wenn ein eindeutiger Gewinn an Performance zu erkennen ist. Ein Shellscript darf nur Befehle und Programme aufrufen die im Lieferumfang von abas oder der zugrundeliegenden Linux Distribution vorhanden sind. Ein Shellscript ist ausreichend zu dokumentieren. Shellskripte dürfen nur nach Freigabe der Teamleads Developing erstellt werden. Eine Alternative zu einem notwendigen Shellscript ist immer ein AJO

Programm. In diesem stehen ähnlich performante Funktionen/Methoden wie in einem Shellskript bereit.

Abweichende Programmiersprachen

Die Verwendung von abweichenden Programmiersprachen (neben FO/FO2/Java/Shellscripten) wie z.B. PHP, Python sind immer mit einem Risiko verbunden. Auch wenn ein Programm sehr lange ohne Fehler läuft wird doch evtl. nach einigen Jahren eine Expertise zur Nacharbeit im Zuge eines Upgrades benötigt. Weiterhin sind diese Programmiersprachen auch immer in direkter Abhängig zum Betriebssystem bzw. der Sprachenbasis (vgl Php 7.1 vs Php 8).

Deswegen sind abweichende Programmiersprachen grundsätzlich nicht erlaubt.

Cronjobs

Grundsätzliche Strategien

Zur zeitpunktgesteuerten Ausführung von Programmen werden Cronjobs verwendet. Da jedes ausgeführte Programm auch zugleich eine Systemlast erzeugt, muss der Abstand zwischen zwei Cronjobläufen genau überprüft werden.

Je nach verarbeiteter Datenmenge und Laufzeit eines FOP ist es wichtig darauf zu achten, dass nicht zwei datenintensive Programme mittels einem Cronjob parallel angestoßen werden. Es muss dabei auch bedacht werden, dass ein FOP im Laufe der Jahre eine steigende Datenmenge verarbeitet und dadurch länger als vorgesehen läuft und damit in Konkurrenz zu einem anderen FOP tritt.

Für ein Cronjob-FOP darf kein Passwort übergeben werden. Das FOP muss immer über einen Benutzer ohne Linux Login ausgeführt werden. (SSO für Cronjob?)

Der Einsatz von incron, systemd, ist nur nach Freigabe durch die Teamleads Developing erlaubt.

Mehrere einzelne Cronjobs

• Vorteile

- Individuelle Zeitsteuerung für jede Aufgabe
- Mehr Kontrolle und Übersichtlichkeit, wenn die Crontab gut gepflegt ist
- Unabhängigkeit: Fehler in einem Job beeinflussen andere nicht
- Flexibilität bei Anpassungen und Erweiterungen

• Nachteile

- Gefahr von Überschneidungen und Sperren (z.B. gleichzeitiger Datenbankzugriff)
- Bei vielen Jobs kann die Crontab unübersichtlich werden
- Dokumentation und Nachvollziehbarkeit müssen sichergestellt werden

Ein zentraler Cronjob mit sequentieller Abarbeitung

• Vorteile

- Gute Übersicht und einfache Dokumentation
- Abhängigkeiten zwischen Aufgaben können direkt abgebildet werden
- Fehlerhandling und Logging zentral steuerbar

• Nachteile

- Weniger flexibel bei der Zeitsteuerung einzelner Aufgaben
- Fehler in einem Schritt können nachfolgende Aufgaben blockieren
- Anpassungen erfordern oft Änderungen am gesamten Ablauf

Beispiel:

Beispiel einer Crontab für einen stündlichen laufenden Cronjob zur Steuerung über ein zentrales Cronjob-FOP

#	Minute	Stunde	Tag-im-Monat	Monat	Wochentag	Befehl
0	*	*	*	*	1-5	eval \$(sh ./denv.sh); owcron/CRONJOB.F02

Erläuterung der Zeitfelder:

- **Minute: 0** (Ausführung jeweils zur vollen Stunde)
- **Stunde: *** (jede Stunde)
- **Tag-im-Monat: *** (jeder Tag)
- **Monat: *** (jeder Monat)
- **Wochentag: 1-5** (Montag bis Freitag)

Beispiel zentrales Steuerfop für obigen Aufruf

```
..aktuelle Stunde beim Start des Programms merken.  
.formula U|xistunde = F|hours(G|currttime)  
switch(U|xistunde)  
  case (5){  
    ..Logik für 5 Uhr wird aufgerufen.  
    .call CronJobAt5oClock()  
  }  
  case (6){  
    ..Logik für 6 Uhr wird aufgerufen.  
    .call CronJobAt6oClock()  
  }  
  case (15){  
    ..Logik für 15 Uhr wird aufgerufen.  
    .call CronJobAt15oClock()  
  }
```

Alternative Ansätze

- Kombination aus zentralen und einzelnen Cronjobs, abhängig von Abhängigkeiten und Anforderungen

Best Practices

- **Dokumentation** Jede Cronjob-Konfiguration sollte dokumentiert werden (Zweck, Zeitplan, Abhängigkeiten, Besonderheiten).
- **Benutzerwahl**
 - Meistens wird ein dedizierter Batch-User verwendet (z.B. **batch**, **berp**, **bdemo**).
 - Root oder Mandantenbenutzer nur in Ausnahmefällen und mit klarer Begründung.
 - Keine zu breite Verteilung auf verschiedene User, um Übersichtlichkeit zu wahren.
- **Fehlerhandling & Logging**
 - Jeder Cronjob sollte ein eigenes Logfile führen, idealerweise mit Timestamps für jede Ausführung.

- Fehlerausgaben und reguläre Ausgaben sollten getrennt und nachvollziehbar protokolliert werden.
- Bei sequentiellen Abläufen: Sicherstellen, dass Fehler in einem Schritt nicht unbemerkt nachfolgende Jobs blockieren.
- **Test & Monitoring**
 - Nach Einrichtung sollten Cronjobs getestet werden.
 - Monitoring einrichten: Wurde der Job ausgeführt? Gab es Fehler? Wann genau?
- **Schulungen & Templates**
 - Bereitstellung von Shell-Skriptvorlagen und Schulungen für die Kollegen, um Best Practices zu fördern.

Empfohlene Vorgehensweise (je nach Szenario)

- **Unabhängige Aufgaben** → Einzelne Cronjobs mit individuellen Zeitplänen
- **Abhängige Aufgaben (Reihenfolge wichtig)** → Zentraler/sequentieller Cronjob oder Shell-Skript, das die Aufgaben nacheinander ausführt
- **Komplexe Anforderungen (z.B. Cloud, Rechte, viele Jobs)** → Kombination aus beiden Ansätzen, ggf. mit externem Scheduler oder systemd/incron

Zusätzliche Hinweise aus der Praxis

- **Pflege der Crontab** Einheitlicher Ort für Cronjobs (z.B. `/etc/crontab` oder `/var/spool/cron`) sorgt für Übersichtlichkeit.
- **Zeitplanung** Überlappungen vermeiden, Laufzeiten beachten, ggf. Sperr-Flags verwenden.
- **Flexibilität** Anpassungen an Kundenbedürfnisse und Systemumgebung sind oft notwendig – keine „One size fits all“-Lösung.

Performance und Datenmengen

Jegliche Customization birgt immer die Gefahr die Benutzung des Systems zu verlangsamen, da zusätzlicher Programmcode durchlaufen wird. Da sich ERP Umgebungen permanent ändern ist es dadurch auch unerlässlich bei jeder Customization auch an zukünftige Erweiterungen zu denken, insbesondere was die Datenmengen im ERP betrifft. Insbesondere bei Selektionen über Stammdaten ist im Laufe der Jahre mit einem stetigen Zuwachs der Datenmengen zu rechnen, während im Bereich der Vorgangsdaten diese meist halbwegs stabil bleiben.

Deshalb sind folgende Überlegungen grundsätzlich bei der Erstellung einer Customization notwendig:

- Welche Anzahl von Datensätzen werden mit der Customization in der Produktivumgebung selektiert?
- Kann ein Zuwachs an Datensätzen entstehen, welche die Selektion verlangsamen und damit die Performance verringern?
- Sind alle verwendeten Selektionen optimal ausgeführt? Werden passende Schlüssel verwendet? Welche Trefferquote wird bei den Selektionen erreicht? Ist die Trefferquote auch bei größerer Datenmenge noch ausreichend?

Benennung von Variablen Datenbank bzw. Infosysteme

Der generelle Aufbau einer Variable ist wie folgt:

[Präfix][Variablenart][Bezeichnung]

	Datenbank bzw. Infosystem
Präfix	Als Präfix werden die ersten beiden Buchstaben der Datenbank genutzt bzw. bei Infosysteme "is". Die 3. Stelle muss ein "y" sein (in Datenbanken zwingend durchs System geprüft, auch bei Infosystemen so nutzen). Tabellenvariablen enthalten dann noch zusätzlich ein "t" an 4. Stelle z.B. evytartikel
Variablenart	Orientiert sich an der Art der Variable. Für eine Textvariable (text) wäre die Variablenart ein „t“ (analog der Auflistung in den Guidelines FO/FO2 für .type/.var): t = Text, i = Integer, r = Real, b = Bool, d = Date, w = Week, p = Pointer bzw. Verweis, a = Aufzählung. Zusätzlich: bu = Buttons, dv = Dateiverweise
weitere Stellen der Variablennamen	Der vollständige Variablenname muss sinnhaft zur Verwendung der Variable sein. Es ist besser, längere Variablennamen zu verwenden, weil sie besser gesucht werden können z.B. ytselect statt ytsel. Variablennamen sollen also wenn möglich nicht abgekürzt werden.
Nomenklatur	nur Kleinbuchstaben, da in der Datenbank Groß- und Kleinschreibung unterschieden wird und das sonst zu Problemen führen kann.

Dont's (was müssen wir vermeiden)

Ungenutzter Programmcode

- **Kein ungenutzter Code im Programm** Nicht verwendete Variablen, Prozeduren oder Funktionsaufrufe müssen entfernt werden.
- **Keine unnötigen Vorlagenteile übernehmen** Aus Programmvorlagen darf nur übernommen werden, was tatsächlich im aktuellen Programm genutzt wird (z. B. keine EDP-Prozeduren, wenn sie nicht aufgerufen werden).
- **Keine Codeabschnitte „auf Vorrat“ behalten** Wenn später Erweiterungen nötig sind, wird der benötigte Code gezielt aus der Vorlage oder bestehenden Schnipseln nachträglich eingefügt – nicht vorab eingebaut.

Sonstige dont's

- **Entwicklung im Entwicklungsmandant, keine Entwicklung im Echt- bzw. Demomandant** Gibt es Projekt einen Entwicklungsmandant, dann müssen Änderungen dort durchgeführt werden. Es darf nicht nur im Echt- bzw. Demomandant geändert werden. Werden Datensätze wie Kurztexte oder auch Aufzählungen, Wertebezeichner etc. benötigt, dann ist darauf zu achten, dass die Nummer in Entwicklungs- und Echtmandant gleich ist.
- **Installation im Echtmandant, die im laufenden Betrieb stören** Generell sind Arbeiten, die zu Performanceeinbußen in Echtmandanten führen können, abzustimmen (beispielsweise: Schlüsselreorganisation, kleine Reorganisation, Masken generieren, Mandantenkopie).
- **Servernamen oder Pfade nicht fix vorgeben sondern über Variablen angeben** Es sind keine festen Servernamen und keine absoluten Pfadangaben zu verwenden. Stattdessen können **E | HOSTNAME**, **G | clwindir**, **\$MANDANTDIR** usw. verwendet werden.